Système de Gestion de Bases de Données

Chapitre 4: Langage SQL sous Oracle

Mme Salma Ben Saïd
Maître Assistant, Département GPI, INSAT
Laboratoire de recherche SITI, ENIT
salma.bensaid@insat.ucar.tn
salma_ben_said@yahoo.fr

Introduction

- **Modèle relationnel**

– Relations (tables): attributs typés (colonnes), n-uplets (lignes)

– Base de données = ensemble de relations

– Schéma relation = nom relation + ensemble attributs avec leurs types

– Schéma BD = ensemble des schémas des relations

- **Algèbre relationnelle**

– Ensemble d'opérateurs agissant sur des relations et produisant des relations

– Sélection, projection, produit cartésien, jointure, différence, union, …

– Requête = composition d'opérateurs algébriques → expression *procédurale*

Ex. Les noms des propriétaires de voitures immatriculées avant 1989

$$\Pi_{\text{Proprietaire}}(\sigma_{\text{Annee}<1989}(\text{VEHICULE}))$$

Introduction

Tout Système de Gestion de Base de Données Relationnelle (SGBDR), dont Oracle, utilise le langage SQL permettant trois principales fonctionnalités :

- Langage de Définition de Données (LDD) : création et modification de la structure des Bases de Données.
- Langage de Manipulation de Données (LMD) : insertion, modification et interrogation des données des Bases de Données.
- Langage de Contrôle des Données (LCD) : gestion de la sécurité, des utilisateurs, de la confidentialité et de l'accès aux données.

SQL = Structured Query Language, est un standard ANSI depuis 1986. Il permet la définition, la manipulation et le contrôle d'une base de données relationnelle. Il se base sur l'algèbre relationnelle.

Partie 1

Langage de Définition des Données (LDD)

- **Création d'une table**
  - Principaux types de données
  - Principales contraintes
- **Modification d'une table**
- **Suppression d'une table**
- **Les contraintes**
  - Clé primaire : PRIMARY KEY
  - Clé étrangère : FOREIGN KEY
  - Non vide : NOT NULL
  - Unique : UNIQUE
  - Intégrité de domaine : CHECK

LDD : **Création d'une table**

CREATE TABLE ....

```
CREATE TABLE NomTable (

nomColonne1  TYPE_DE_DONNEES [CONSTRAINT nomContrainte TYPE_CONTRAINTE  [...]],

nomColonne2  TYPE_DE_DONNEES [CONSTRAINT nomContrainte TYPE_CONTRAINTE  [...]],

nomColonne3  TYPE_DE_DONNEES,

nomColonne4  TYPE_DE_DONNEES  DEFAULT valeur par défaut ,

....

 [CONSTRAINT nomContrainte TYPE_CONTRAINTE (nomColonneX,..., nomColonneY)  ]

) ;
```

Séparateur

Fin de la commande SQL

LDD : **Création d'une table**

Principaux types de données :

TYPE_DE_DONNEES		Syntaxe	Description
Entier		NUMBER(n)	Entier composé de n chiffres au maximum. n≤38
Réel	à virgule flottante	NUMBER	
	à virgule fixe	NUMBER(s , p)	Réel composé de s chiffres dont p chiffres après la virgule. NUMBER(5,2) --> 999.99
Chaîne de caractères	De taille fixe	CHAR(n)	ch de c de taille exactement n caractères (complété par des espaces) . 1≤n≤2000
	De taille variable	VARCHAR2(n)	Ch de c de taille au maximum n caractères. (ajusté au contenu). 1≤n≤4000
Date		DATE	Selon le format utilisé : '--/--/----'

Principales contraintes :

TYPE_CONTRAINTE	nomContrainte
PRIMARY KEY	PK_NomTable
FOREIGN KEY	FK_NomTable1_NomTable2
NOT NULL	NN_nomColonne
UNIQUE	UN_nomColonne
CHECK	CHK_nomColonne

LDD : **Création d'une table**

Soit le schéma relationnel suivant :

Client (numClient, nomClient, prenomClient, adresse, datNaiss)

Commande (numCommande, datCde, #numClt)

LigneCommande (#numCde, #numArticle, qteCdee)

Article (numArticle, designation, prixHT, couleur)

Refus (#numCde, #numArticle, datRefus, qte_Refusee, motif)

Tableau descriptif des données :

numClient, numClt	Numérique de8 chiffres maximum
nomClient, prenomClient, adresse, designation, couleur, motif	Chaine de longueur 50 caractères au maximum
numCommande, numCde, numArticle	Numérique de 6 chiffres maximum
qte_Cdee, qte_Refusee	Numérique de 3 chiffres maximum
prixHT	Réel, champs obligatoirement renseigné
datNaiss, datCde, datRefus	Date / datCde, datRefus: par défaut la date du jour

LDD : **Création d'une table**

Exemple :

```
CREATE TABLE Client (
numClient   NUMBER(8) CONSTRAINT PK_Client PRIMARY KEY,
nomClient   VARCHAR2(50),
prenomClient VARCHAR2(50),
adresse VARCHAR2(50),
datNaiss DATE ) ;
```

LDD : **Modification d'une table**

ALTER TABLE nomTable **... ;**

Ajouter une colonne ou plusieurs colonnes :

ALTER TABLE nomTable **ADD (** nomColonneA TYPE_DE_DONNEES [...], ..., nomColonneZ TYPE_DE_DONNEES [...] **) ;**

Supprimer une colonne :

ALTER TABLE nomTable **DROP COLUMN** nomColonne **CASCADE CONSTRAINTS ;**

CASCADE CONSTRAINTS**:** Option à ajouter pour forcer la suppression d'une colonne référence**.**

Renommer une colonne :

ALTER TABLE nomTable **RENAME COLUMN** nomColonne **TO** nouveauNomColonne **;**

Modifier type de données d'une colonne :

ALTER TABLE nomTable **MODIFY** nomColonne **NouveauTypedeDonnées ;**

LDD : **Modification d'une table**

ALTER TABLE nomTable **... ;**

Ajouter une Contrainte :

ALTER TABLE nomTable **ADD CONSTRAINT** nomContrainte **TYPE_CONTRAINTE**
(nomColonne1,.., nomColonneN**);**

Remarque: nomColonne1,.., nomColonneN:
sont les colonnes qui portent la contrainte.

Supprimer une contrainte :

ALTER TABLE nomTable **DROP CONSTRAINT** nomContrainte **;**

Renommer une table :

ALTER TABLE nomTable **RENAME TO** NouveauNomTable **;**

ou bien

RENAME nomTable **TO** NouveauNomTable **;**

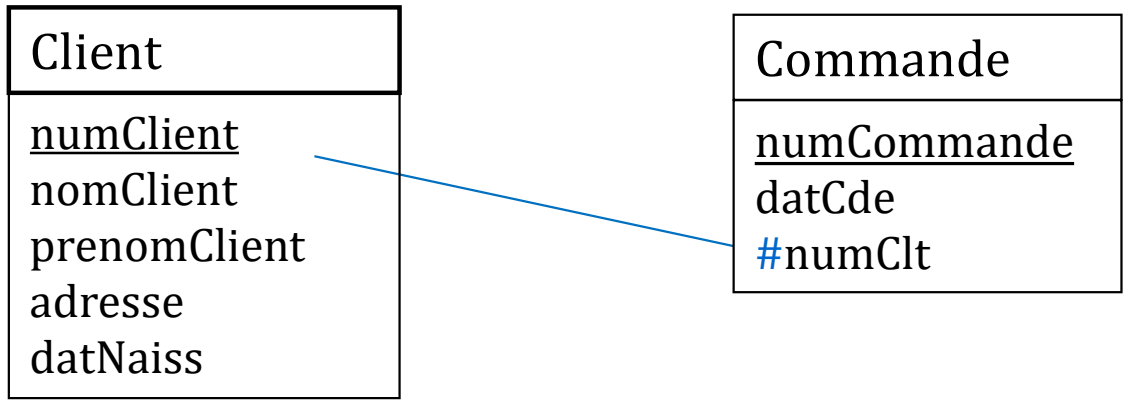
LDD : **Suppression d'une table**

DROP TABLE nomTable **;**

DROP TABLE nomTable **CASCADE CONSTRAINTS ;**

Remarque: CASCADE CONSTRAINTS **:** Option à ajouter pour forcer la suppression de la table au cas où elle sert de référence**.** Ceci en supprimant la contrainte référentielle qui les relie puis supprimer la table concernée,

Exemple :



DROP TABLE Client **CASCADE CONSTRAINTS ;**

LDD : **Les contraintes**

Définition des contraintes :

Les contraintes peuvent être définies de trois manières différentes, selon leur emplacement de définition:

(1) contrainte colonne: définie lors de la définition de la colonne concernée.

(2) contrainte table: définie après la définition des colonnes et avant fermeture de la parenthèse de la commande CREATE TABLE.

(3) contrainte ajoutée: définie à l'aide de la commande ALTER TABLE.

```
CREATE TABLE Nom_Table (
nomColonne1   TYPE_DE_DONNEES CONSTRAINT nomContrainte TYPE_CONTRAINTE ,
CONSTRAINT nomContrainte TYPE_CONTRAINTE ( nomColonneX) );
```

```
ALTER TABLE nomTable ADD CONSTRAINT nomContrainte TYPE_CONTRAINTE
(nomColonne1,.., nomColonneN);
```

LDD : **Les contraintes**

Clé Primaire: PRIMARY KEY : Toute table ne contient qu'une seule clé primaire. Elle peut être simple ou composite:

PK simple: portée par une seule colonne: **(1) ou (2) ou (3)**

```
CONSTRAINT PK_NomTable PRIMARY KEY
```

PK composite: portée par deux colonnes ou plus: **(2) ou (3)**

```
CONSTRAINT PK_NomTable PRIMARY KEY(nomCol1,..,nomColn)
```

Clé étrangère : FOREIGN KEY :

FK simple ou composite: définie comme : **(2) ou (3)**

```
CONSTRAINT FK_TableFille_ TableMère FOREIGN KEY( nomColonneFK )
REFERENCES TableMère( nomcolonnePK) [ ON DELETE CASCADE ]
```

Optionnelle: pour supprimer les enregistrements fils lors de la suppression d'un parent.

LDD : **Les contraintes**

Exemple :

```
CREATE TABLE Commande (
numCommande NUMBER(6) CONSTRAINT PK_Commande PRIMARY KEY,
datCde      DATE      DEFAULT    SYSDATE,
numClt   NUMBER(8) ,
CONSTRAINT FK_Commande_Client FOREIGN KEY( numClt ) REFERENCES Client( numClient ) );
```

Fonction qui retourne la date du système, définie comme valeur par défaut.

```
CREATE TABLE Article (
numArticle NUMBER(6) CONSTRAINT PK_Article PRIMARY KEY,
designation VARCHAR2(100),
couleur VARCHAR2(30),
prixHT NUMBER );
```

LDD : **Les contraintes**

Exemple : clé primaire composite

```
CREATE TABLE LigneCommande (
numCde NUMBER(6),
numArticle NUMBER(6) ,
qteCdee   NUMBER(3) ,
CONSTRAINT PK_LigneCommande PRIMARY KEY( numCde,numArticle ) ,
CONSTRAINT FK_LigneCde_Commande FOREIGN KEY( numCde ) REFERENCES
Commande( numCommande ),
CONSTRAINT FK_LigneCde_Article FOREIGN KEY( numArticle ) REFERENCES Article(
numArticle ) );
```

LDD : **Les contraintes**

Exemple : clé étrangère composite

```
CREATE TABLE Refus  (
numCmde NUMBER(6),
numArticle NUMBER(6),
datRefus DATE DEFAULT SYSDATE,
qteRefusee NUMBER(3),
motif VARCHAR2(200),
CONSTRAINT PK_Refus PRIMARY KEY(numCmde,numArticle,datRefus),
CONSTRAINT FK_Refus_LigneCmde FOREIGN KEY(numCmde,numArticle)
REFERENCES LigneCommande(numCmde,numArticle) );
```

LDD : **Les contraintes**

Non vide : NOT NULL

NOT NULL : ne peut être définie que comme contrainte colonne **(1),** c'est-à-dire lors de la définition de la colonne concernée. Aussi, lors de sa modification avec la commande MODIFY …. Et avant toute insertion de données,

Exemple :
ALTER TABLE Client MODIFY adresse CONSTRAINT NN_adresse NOT NULL ;

Unicité : UNIQUE

UNIQUE : peut être définie selon **(1) ou (2) ou (3)**
Exemple :
ALTER TABLE Article ADD CONSTRAINT UN_designation UNIQUE(designation);

LDD : **Les contraintes**

Intégrité de domaine : CHECK

CHECK : peut être définie selon : **(1) ou (2) ou (3)**

Définie la plage de valeurs dans lesquelles seront insérées les données au niveau de la colonne concernée.

nomColonne **opérateur de comparaison** **valeur(s)**

< , <= , >, >=, = , <>,

NOT / IN (v1, v2, v3)

NOT / BETWEEN b1 AND b2 avec b1 < b2

NOT LIKE '…'

AND / OR

LDD : **Les contraintes**

Intégrité de domaine : CHECK

```
CREATE TABLE Etudiant (

.....

genre CHAR CONSTRAINT CHK_genre CHECK(genre IN ( 'F', 'f', 'M', 'm' ))

ou

genre CHAR CONSTRAINT CHK_genre CHECK(UPPER(genre) IN ( 'F','M' ))

ou

genre CHAR CONSTRAINT CHK_genre CHECK(LOWER(genre) IN ( 'f', 'm' ))
    ....);
```

```
ALTER TABLE Article ADD CONSTRAINT CHK_prixHT CHECK(prixHT BETWEEN
150 AND 560 ) ;

ou

ALTER TABLE Article ADD CONSTRAINT CHK_prixHT CHECK(prixHT >= 150 AND
prixHT <=  560 ) ;
```

Exercice :

Soit le schéma relationnel suivant :

Employe (idEmp, nom, d_Nais, adresse, grade, salaire, #idSup, #numDept)
Departement (numDept, nomDept, # idResp)

Avec :

idEmp, idSup, idResp:	Identifiant de l'employé/ Entier sur 8 chiffres au maximum
nom :	Nom de l'employé/ Chaîne de longueur 40 caractères maximum
d_Nais:	Date de naissance/ Date
adresse	Adresse de l'employé/ Chaîne de longueur 100 caractères maximum
grade :	Grade de l'employé/ Chaîne de longueur 50 caractères maximum
salaire :	Salaire de l'employé/ réel à virgule flottante
numDept :	Numéro du département / Numérique de 2 chiffres maximum
nomDept :	Nom du département / Chaîne de longueur 60 caractères maximum

1. Créer chacune des tables "Employe" et "Departement" en respectant les types des champs indiqués ci-dessus et en spécifiant toutes les contraintes

```sql
CREATE TABLE Employe (
idEmp NUMBER(8) CONSTRAINT PK_Employe PRIMARY KEY,
nom VARCHAR2(40),
d_Nais DATE,
adresse VARCHAR2(100),
grade VARCHAR2(50),
salaire NUMBER,
idSup NUMBER(8),
CONSTRAINT FK_Emp_Emp FOREIGN KEY(idSup) REFERENCES Employe(idEmp),
numDept NUMBER(2));
```

```sql
CREATE TABLE Departement (
numDept NUMBER(2) CONSTRAINT PK_Departement PRIMARY KEY,
nomDept VARCHAR2(60),
idResp NUMBER(8),
CONSTRAINT FK_Dept_Emp FOREIGN KEY(idResp) REFERENCES Employe(idEmp));
-- Ajout de la contrainte
ALTER TABLE Employe ADD CONSTRAINT FK_Emp_Dept FOREIGN KEY(numDept)
REFERENCES Departement(numDept);
```

```sql
 -- 2 Renommer la colonne nom à nomEmp
ALTER TABLE Employe RENAME COLUMN nom TO nomEmp;
-- 3 Ajouter une colonne prenomEmp de type Ch.c.v de taille 40.
ALTER TABLE Employe ADD prenomEmp VARCHAR2(40);
-- 4 salaire doit être compris entre 1700 et 7000
ALTER TABLE Employe ADD CONSTRAINT CHK_salaire CHECK (salaire BETWEEN 1700 AND 7000);
-- 5 Modifier type de grade à 1 c. obligatoirement renseigné
-- et avoir une des valeurs suivantes ; A, B ou C.
ALTER TABLE Employe MODIFY grade CHAR CONSTRAINT NN_grade NOT NULL ;
ALTER TABLE Employe ADD CONSTRAINT CHK_grade CHECK(UPPER(grade) IN('A','B','C'));
-- ou
ALTER TABLE Employe MODIFY grade CHAR CONSTRAINT NN_grade NOT NULL
CONSTRAINT CHK_grade CHECK(UPPER(grade) IN('A','B','C'));
```

Exercice :

Soit le schéma relationnel de la base de données «Etudiant_Enseignant_Matiere_Evaluation» :

Etudiant (numEtu, nomEtu, prenomEtu, age, genre)
Enseignant (numEns, nomEns, prenomEns, grade)
Matiere (numMat, nomMat, coefficient, #numEns)
Evaluation (#numEtu, #numMat, note)

Avec :

NumEtu :	N° de l'étudiant/ Numérique de 6 chiffres maximum
NomEtu :	Nom de l'étudiant/ Chaîne de longueur 20 caractères maximum
PrenomEtu :	Prénom de l'étudiant/ Chaîne de longueur 20 caractères maximum
Age :	Age de l'étudiant, Numérique de 2 chiffres maximum
genre :	genre de l'étudiant {'M', 'F'} où ; 'M' : Masculin, 'F': Feminin/ Chaîne de longueur 1 caractère exactement
NumEns :	Numéro de l'enseignant/ Numérique de 6 chiffres maximum
NomEns :	Nom de l'enseignant/ Chaîne de longueur 30 caractères maximum
PrenomEns :	Prénom de l'enseignant/ Chaîne de longueur 30 caractères maximum
Grade :	Grade de l'enseignant {'ASS', 'MCF', 'PRO'} où ; 'ASS' : Assistant, 'MCF' : Maître de conférence, 'PRO' : Professeur/ Chaîne de longueur 3 caractères exactement
NumMat :	N° de la matière/ Numérique de 4 chiffres maximum
NomMat :	Nom de la matière/ Chaîne de longueur 25 caractères maximum
Coefficient :	Coefficient de la matière/ Numérique de 1 chiffre exactement
Note :	Note de l'étudiant dans une matière donnée/ Numérique à virgule fixe / compris entre 0 et 20.